T10 Aprendizaje profundo

Índice

1. Introducción
2. LeNet
3. Capas convolucionales
4. Pooling
5. ImageNet
6. AlexNet
7. Dropout
8. Inception
9. ResNet
10. Capas de normalización
11. Transformer
12. Vision Transformer

1 Introducción

Aprendizaje profundo: área del ML especializada en el aprendizaje de redes con (muchas) capas ocultas

Revolución del aprendizaje profundo: el aprendizaje profundo alcanza gran popularidad a partir de los 2010, principalmente por la mejora de resultados en reconocimiento automático del habla (2011) y clasificación de imágenes (2012)

Objetivo: introducir arquitecturas destacadas del aprendizaje profundo para clasificación de imágenes principalmente

LeNet (1998): red **convolucional (CNN)** pionera, introducida en 1998 por Yann LeCun para MNIST

- Propone el uso de capas convolucionales y pooling (agrupación)

AlexNet (2012): CNN de Alex Krizhevsky para ImageNet que atrajo mucho interés

- Resultados espectaculares gracias a la profundidad, uso de GPUs, ReLUs y dropout

Inception (2014): CNN de Google que introduce la meta-arquitectura típica de las CNNs modernas

ResNet (2015): CNN de Microsoft que introduce **conexiones residuales** para entrenar modelos muy profundos

- Propone el uso de conexiones residuales y añade capas de normalización

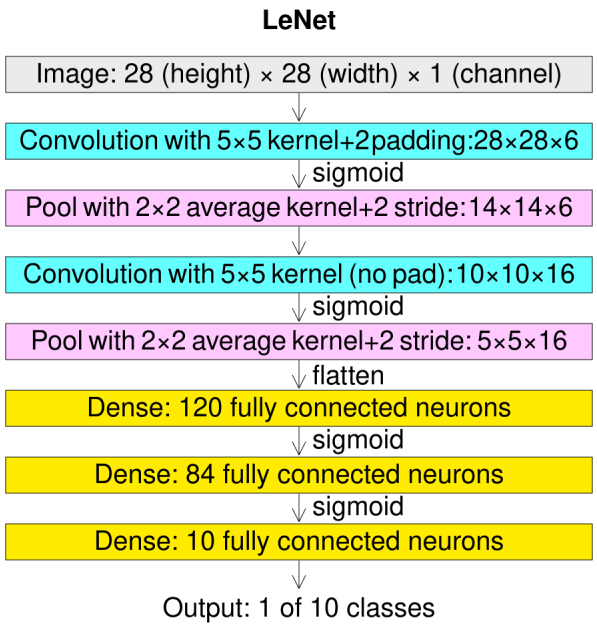
Transformer (2017): arquitectura de Google para traducción automática basada en el **mecanismo de atención**

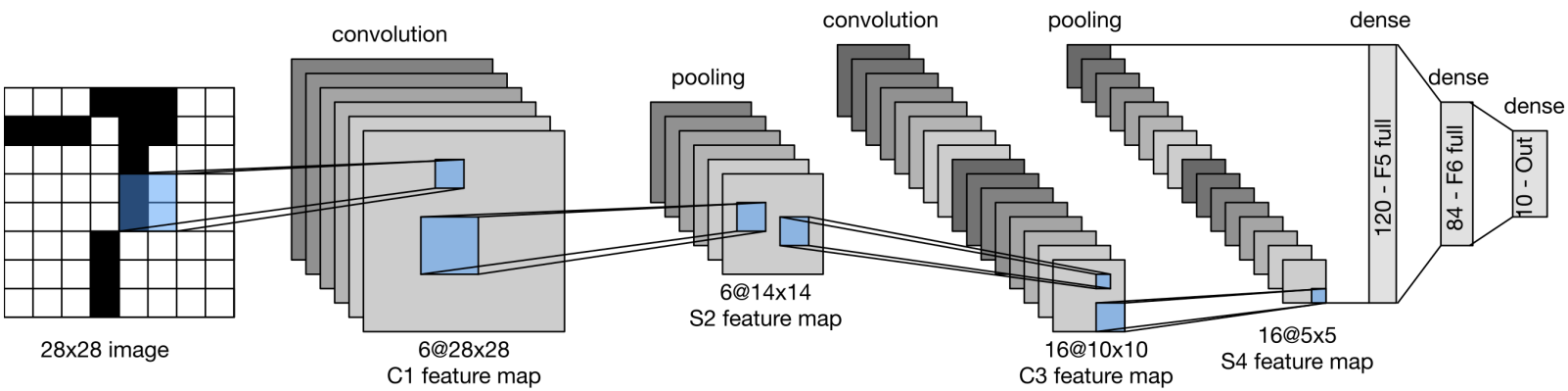
- Se considera la base de los modelos IA actuales más competitivos

Vision Transformer (2020): adaptación de Transformer para visión

2 LeNet

LeNet (1998): red **convolucional (CNN)** pionera, introducida en 1998 por Yann LeCun para MNIST





Experimento con MNIST: con relus en lugar de sigmoides

In [ ]:
```
import torch; import torch.nn as nn; import torch.nn.functional as F
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5)
        self.fc1 = nn.Linear(16 * 4 * 4, 120); self.fc2 = nn.Linear(120, 84); self.fc3 = nn.Linear(84, 10)
    def forward(self, x):
        x = F.max_pool2d(F.relu(self.conv1(x)), kernel_size=2)
        x = F.max_pool2d(F.relu(self.conv2(x)), kernel_size=2)
        x = torch.flatten(x, 1); x = F.relu(self.fc1(x)); x = F.relu(self.fc2(x)); x = self.fc3(x)
        return x
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
model = LeNet().to(device); print(device, model)
```

```
cuda LeNet(
  (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (fc1): Linear(in_features=256, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

In [ ]:
```
import numpy as np; import datasets
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy"); ds
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 1, 28, 28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 1, 28, 28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
X = torch.Tensor(X_train).to(device); y = torch.Tensor(y_train).to(device, dtype=torch.long)
loss_fn = nn.CrossEntropyLoss(); optimizer = torch.optim.AdamW(model.parameters(), lr=1e-2)
model.train(); epochs = 200
for t in range(epochs):
    pred = model(X); loss = loss_fn(pred, y); loss.backward(); optimizer.step(); optimizer.zero_grad()
X = torch.Tensor(X_test).to(device); pred = model(X).detach().cpu().numpy()
acc = (pred.argmax(1) == y_test).sum().item() / y_test.size; print(f'Precisión {acc:.2%}')
```

```
Precisión 98.72%
```

3 Capas convolucionales

Valid convolution: $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X}$, con un filtro $\mathbf{W}$ $f_h \times f_w$ de una imagen $\mathbf{X}$ $x_h \times x_w$

$$\mathbf{Y} = \begin{bmatrix} w_{0,0} & \cdots & w_{0,f_w-1} \\ \vdots & \ddots & \vdots \\ w_{f_h-1,0} & \cdots & w_{f_h-1,f_w-1} \end{bmatrix} \circledast \begin{bmatrix} x_{0,0} & \cdots & x_{0,x_w-1} \\ \vdots & \ddots & \vdots \\ x_{x_h-1,0} & \cdots & x_{x_h-1,x_w-1} \end{bmatrix} = \begin{bmatrix} y_{0,0} & \cdots & y_{0,y_w-1} \\ \vdots & \ddots & \vdots \\ y_{y_h-1,0} & \cdots & y_{y_h-1,y_w-1} \end{bmatrix}$$

$$y_{ij} = \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} w_{u,v}\, x_{i+u,j+v} \qquad \begin{array}{ll} i = 0, \ldots, y_h - 1 & y_h = x_h - f_h + 1 \\ j = 0, \ldots, y_w - 1 & y_w = x_w - f_w + 1 \end{array}$$

Ejemplo: $f_h = f_w = 2$, $x_h = x_w = 3$, por lo que $y_h = y_w = 2$

$$\begin{bmatrix} w_{00} & w_{01} \\ w_{10} & w_{11} \end{bmatrix} \circledast \begin{bmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{bmatrix} = \begin{bmatrix} y_{00} & y_{01} \\ y_{10} & y_{11} \end{bmatrix}$$

$$\begin{aligned} y_{00} &= w_{00}x_{00} + w_{01}x_{01} + w_{10}x_{10} + w_{11}x_{11} \\ y_{01} &= w_{00}x_{01} + w_{01}x_{02} + w_{10}x_{11} + w_{11}x_{12} \\ y_{10} &= w_{00}x_{10} + w_{01}x_{11} + w_{10}x_{20} + w_{11}x_{21} \\ y_{11} &= w_{00}x_{11} + w_{01}x_{12} + w_{10}x_{21} + w_{11}x_{22} \end{aligned}$$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \circledast \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

In [ ]:
```
import numpy as np; import torch
W = torch.nn.Conv2d(1, 1, 2, bias=False)
W.weight.data = torch.tensor([[ [[1.,1.], [1.,1.]] ]], requires_grad=True)
X = torch.tensor([[ [[1,1,0], [0,1,0], [0,1,1]] ]], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[3., 2.],          [2., 3.]]]])
```

Zero-padding: relleno cero $2\,p_h \times 2\,p_w$, por lo que $y_h = (x_h + 2p_h - f_h + 1)$ e $y_w = (x_w + 2p_w - f_w + 1)$

Same convolution: zero-padding con $2p_h = f_h - 1$ y $2p_w = f_w - 1$ para conservar el tamaño de la entrada

Ejemplo: $f_h = f_w = 3,\ x_h = x_w = 3,\ p_h = p_w = 1$ por lo que $y_h = y_w = 3$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \circledast \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 & 2 \\ 4 & 5 & 4 \\ 2 & 3 & 3 \end{bmatrix}$$

In [ ]:
```
W = torch.nn.Conv2d(1, 1, 2, padding='same', bias=False)
W.weight.data = torch.tensor([[ [[1.,1.,1.], [1.,1.,1.], [1.,1.,1.]] ]], requires_grad=True)
X = torch.tensor([ [[[1,1,0], [0,1,0], [0,1,1]] ]], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[3., 3., 2.],          [4., 5., 4.],          [2., 3., 3.]]]])
```

Strided convolution: a saltos s_h y s_w, luego $y_h = \left\lfloor \dfrac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor$ e $y_w = \left\lfloor \dfrac{x_w + 2p_w - f_w + s_w}{s_w} \right\rfloor$

Ejemplo: $f_h = f_w = 2,\ x_h = 2,\ x_w = 6,\ p_h = p_w = 0,\ s_h = s_w = 2,$ por lo que $y_h = 1$ e $y_w = 3$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \circledast \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 1 & 2 \end{bmatrix}$$

In [ ]:
```
W = torch.nn.Conv2d(1, 1, 2, stride=2, padding=0, bias=False)
W.weight.data = torch.tensor([[ [[1.,1.], [1.,1.]] ]], requires_grad=True)
X = torch.tensor([[ [[1,1,0,1,0,0], [0,1,0,0,1,1]] ]], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[3., 1., 2.]]]])
```

Múltiples canales de entrada: C canales, filtro $\mathbf{W}\ C \times f_h \times f_w$, imagen $\mathbf{X}\ C \times x_h \times x_w$

$$\mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \sum\nolimits_c \mathbf{W}_{c,:,:} \circledast \mathbf{X}_{c,:,:}$$

$$\left[\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}, \begin{bmatrix}2 & 2\\2 & 2\end{bmatrix}\right] \circledast \left[\begin{bmatrix}1 & 1 & 0\\0 & 1 & 0\\0 & 1 & 1\end{bmatrix}, \begin{bmatrix}1 & 0 & 1\\1 & 1 & 1\\0 & 0 & 1\end{bmatrix}\right] = \begin{bmatrix}9 & 8\\6 & 9\end{bmatrix}$$

In [ ]:
```
W = torch.nn.Conv2d(2, 1, 2, bias=False)
W.weight.data = torch.tensor([ [ [[1.,1.],[1.,1.]], [[2.,2.],[2.,2.]] ] ], requires_grad=True)
X = torch.tensor([ [ [[1,1,0],[0,1,0],[0,1,1]], [[1,0,1],[1,1,1],[0,0,1]] ] ], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[9., 8.],          [6., 9.]]]])
```

Múltiples canales de entrada y salida: C de entrada y D de salida, $\mathbf{W}\ D \times C \times f_h \times f_w$, $\mathbf{X}\ C \times x_h \times x_w$

$$\mathbf{Y}_{d,:,:} = \mathbf{W}_{d,:,:,:} \circledast \mathbf{X}$$

$$\left[\left[\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}, \begin{bmatrix}2 & 2\\2 & 2\end{bmatrix}\right], \left[\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}, \begin{bmatrix}1 & 1\\1 & 1\end{bmatrix}\right]\right] \circledast \left[\begin{bmatrix}1 & 1 & 0\\0 & 1 & 0\\0 & 1 & 1\end{bmatrix}, \begin{bmatrix}1 & 0 & 1\\1 & 1 & 1\\0 & 0 & 1\end{bmatrix}\right] = \left[\begin{bmatrix}9 & 8\\6 & 9\end{bmatrix}, \begin{bmatrix}6 & 5\\4 & 6\end{bmatrix}\right]$$

In [ ]:
```
W = torch.nn.Conv2d(2, 2, 2, bias=False)
W.weight.data = torch.tensor([ [ [[1.,1.],[1.,1.]], [[2.,2.],[2.,2.]] ],
                               [ [[1.,1.],[1.,1.]], [[1.,1.],[1.,1.]] ] ], requires_grad=True)
X = torch.tensor([ [ [[1,1,0],[0,1,0],[0,1,1]], [[1,0,1],[1,1,1],[0,0,1]] ] ], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[9., 8.],          [6., 9.]],         [[6., 5.],          [4., 6.]]]])
```

Sesgos: por cada salida d puede añadirse un parámetro de sesgo b_d que se suma a cada elemento de la salida

Convolución 1x1: $f_h = f_w = 1$ para pasar de C a D canales **punto a punto**

Reducción a productos matriciales: unfolding, matrix multiplication and folding

$$\mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \mathrm{Fold}(\mathrm{Unfold}(\mathbf{X})\,\mathrm{vec}(\mathbf{W}))$$

Ejemplo: $\mathbf{X} = \left[\begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \right]$ $\quad \mathbf{W} = \left[\begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix} \right]$ $\quad \mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \begin{bmatrix} 19 \\ 25 \end{bmatrix}$

$$\mathrm{Unfold}(\mathbf{X}) = \begin{bmatrix} 0 & 1 & 3 & 4 \\ 1 & 2 & 4 & 5 \end{bmatrix} \qquad \mathrm{vec}(\mathbf{W}) = \begin{bmatrix} 0 & 1 & 2 & 3 \end{bmatrix}^t \qquad \mathrm{Fold}\left(\begin{bmatrix} 19 \\ 25 \end{bmatrix} \right) = \begin{bmatrix} 19 \\ 25 \end{bmatrix}$$

In [ ]:
```python
X = torch.tensor([[ [[0], [1], [2]], [[3], [4], [5]] ]], dtype=torch.float)
print('X =', X.shape, str(X).replace('\n','').replace(' ',''))
W = torch.nn.Conv2d(2, 1, (1, 2), bias=False)
W.weight.data = torch.tensor([ [ [[0],[1]], [[2],[3]] ] ], requires_grad=True, dtype=torch.float)
print('W =', W.weight.data.shape, str(W.weight.data).replace('\n','').replace(' ',''))
YConv2d = W(X)
print('YConv2d =', YConv2d.shape, str(YConv2d).replace('\n','').replace(' ',''))
Xu = torch.nn.functional.unfold(X, kernel_size=(2, 1)).transpose(1,2) # unfold devuelve transpuesta
print('Xu =', Xu.shape, str(Xu).replace('\n','').replace(' ',''))
vecW = W.weight.data.reshape(-1, 1)
print('vec(W) =', vecW.shape, str(vecW).replace('\n','').replace(' ',''))
Yu = Xu @ vecW
print('Yu =', Yu.shape, str(Yu).replace('\n','').replace(' ',''))
Y = torch.nn.functional.fold(Yu , (2, 1), (2, 1))
print('Y =', Y.shape, str(Y).replace('\n','').replace(' ',''))
(YConv2d - Y).abs().max().item()
```

```
X = torch.Size([1, 2, 3, 1]) tensor([[[[0.],[1.],[2.]],[[3.],[4.],[5.]]]])
W = torch.Size([1, 2, 2, 1]) tensor([[[[0.],[1.]],[[2.],[3.]]]])
YConv2d = torch.Size([1, 1, 2, 1]) tensor([[[[19.],[25.]]]],grad_fn=<ConvolutionBackward0>)
Xu = torch.Size([1, 2, 4]) tensor([[[0.,1.,3.,4.],[1.,2.,4.,5.]]])
vec(W) = torch.Size([4, 1]) tensor([[0.],[1.],[2.],[3.]])
Yu = torch.Size([1, 2, 1]) tensor([[[19.],[25.]]])
Y = torch.Size([1, 1, 2, 1]) tensor([[[[19.],[25.]]]])
```

Out[ ]:
```
0.0
```

4 Pooling

Max pooling: $\mathbf{Y} = \mathrm{MaxPool2d}(\mathbf{X}),$ con un tamaño de filtro $f_h \times f_w$ de una imagen $\mathbf{X}$ $C \times x_h \times x_w$

$$\mathbf{Y}_{c,:,:} = \begin{bmatrix} y_{c,0,0} & \cdots & y_{c,0,f_w-1} \\ \vdots & \ddots & \vdots \\ y_{c,f_h-1,0} & \cdots & y_{c,f_h-1,f_w-1} \end{bmatrix} \qquad y_{cij} = \max_{u=0}^{f_h-1} \max_{v=0}^{f_w-1} x_{c,i+u,j+v} \qquad \begin{matrix} i = 0, \ldots, y_h - 1 \\ j = 0, \ldots, y_w - 1 \end{matrix} \qquad \begin{matrix} y_h = x_h - f_h \\ y_w = x_w - f_w \end{matrix}$$

Ejemplo: $\mathbf{X} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} \qquad f_h \times f_w = (2,2) \qquad \mathbf{Y} = \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}$

In [ ]:
```
import numpy as np; import torch
P = torch.nn.MaxPool2d(2, stride=1)
X = torch.tensor([[ [[0,1,2], [3,4,5], [6,7,8]] ]], dtype=torch.float)
print(str(P(X).data).replace('\n','').replace(' ',''))
```

```
tensor([[[[4.,5.],[7.,8.]]]])
```

Average pooling: $\mathbf{Y} = \mathrm{AvgPool2d}(\mathbf{X}),$ sustituye máximo por media

Ejemplo: $\mathbf{X} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} \qquad f_h \times f_w = (2,2) \qquad \mathbf{Y} = \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$

In [ ]:
```
P = torch.nn.AvgPool2d(2, stride=1)
print(str(P(X).data).replace('\n','').replace(' ',''))
```

```
tensor([[[[2.,3.],[5.,6.]]]])
```

Global max pooling:

$$\mathbf{Y}_c = \text{GlobMaxPool2d}(\mathbf{X})_c = \max_{u=0,\ldots,f_h-1} \max_{v=0,\ldots,f_w-1} \text{MaxPool2d}(\mathbf{X})_{c,u,v}$$

Ej.: $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \end{bmatrix} \quad f_h \times f_w = (2,1) \quad \text{MaxPool2d}(\mathbf{X}) = \begin{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 4 \\ 5 \end{bmatrix} \end{bmatrix} \quad \mathbf{Y} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$

In [ ]:
```
P1 = torch.nn.MaxPool2d((2, 1), stride=1)
X = torch.tensor([[ [[0], [1], [2]], [[3], [4], [5]] ]], dtype=torch.float)
P1X = P1(X); print('MaxPool2d =', P1X.shape, str(P1X.data).replace('\n','').replace(' ',''))
Y, _ = P1X.squeeze(3).max(2); print('GlobMaxPool2d =', Y)
```

```
MaxPool2d = torch.Size([1, 2, 2, 1]) tensor([[[[1.],[2.]],[[4.],[5.]]]])
GlobMaxPool2d = tensor([[2., 5.]])
```

Global average pooling:

$$\mathbf{Y}_c = \text{GlobAvgPool2d}(\mathbf{X})_c = \frac{1}{f_h f_w} \sum_{u=0,\ldots,f_h-1} \sum_{v=0,\ldots,f_w-1} \text{AvgPool2d}(\mathbf{X})_{c,u,v}$$

Ej.: $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \end{bmatrix} \quad f_h \times f_w = (2,1) \quad \text{AvgPool2d}(\mathbf{X}) = \begin{bmatrix} \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}, \begin{bmatrix} 3.5 \\ 4.5 \end{bmatrix} \end{bmatrix} \quad \mathbf{Y} = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$

In [ ]:
```
P1 = torch.nn.AvgPool2d((2, 1), stride=1); P2 = torch.nn.AdaptiveAvgPool2d(1)
X = torch.tensor([[ [[0], [1], [2]], [[3], [4], [5]] ]], dtype=torch.float)
P1X = P1(X); print('AvgPool2d =', P1X.shape, str(P1X.data).replace('\n','').replace(' ',''))
Y = P2(P1X); print('GlobAvgPool2d =', Y.shape, str(Y.data).replace('\n','').replace(' ',''))
```

```
AvgPool2d = torch.Size([1, 2, 2, 1]) tensor([[[[0.5000],[1.5000]],[[3.5000],[4.5000]]]])
GlobAvgPool2d = torch.Size([1, 2, 1, 1]) tensor([[[[1.]],[[4.]]]])
```

5 ImageNet

ImageNet: dataset de $14\,197\,122$ imágenes y $21\,841$ synsets; muy popular desde los 2010

ImageNet en Wikipedia: historia, dataset, challenge y sesgos

ImageNet en HF: ILSVRC/imagenet-1k, gmongaras/Imagenet21K, benjamin-paine/imagenet-1k-256x256, etc.

ImageNet-1K: subconjunto de 1.4M imágenes y 1000 clases

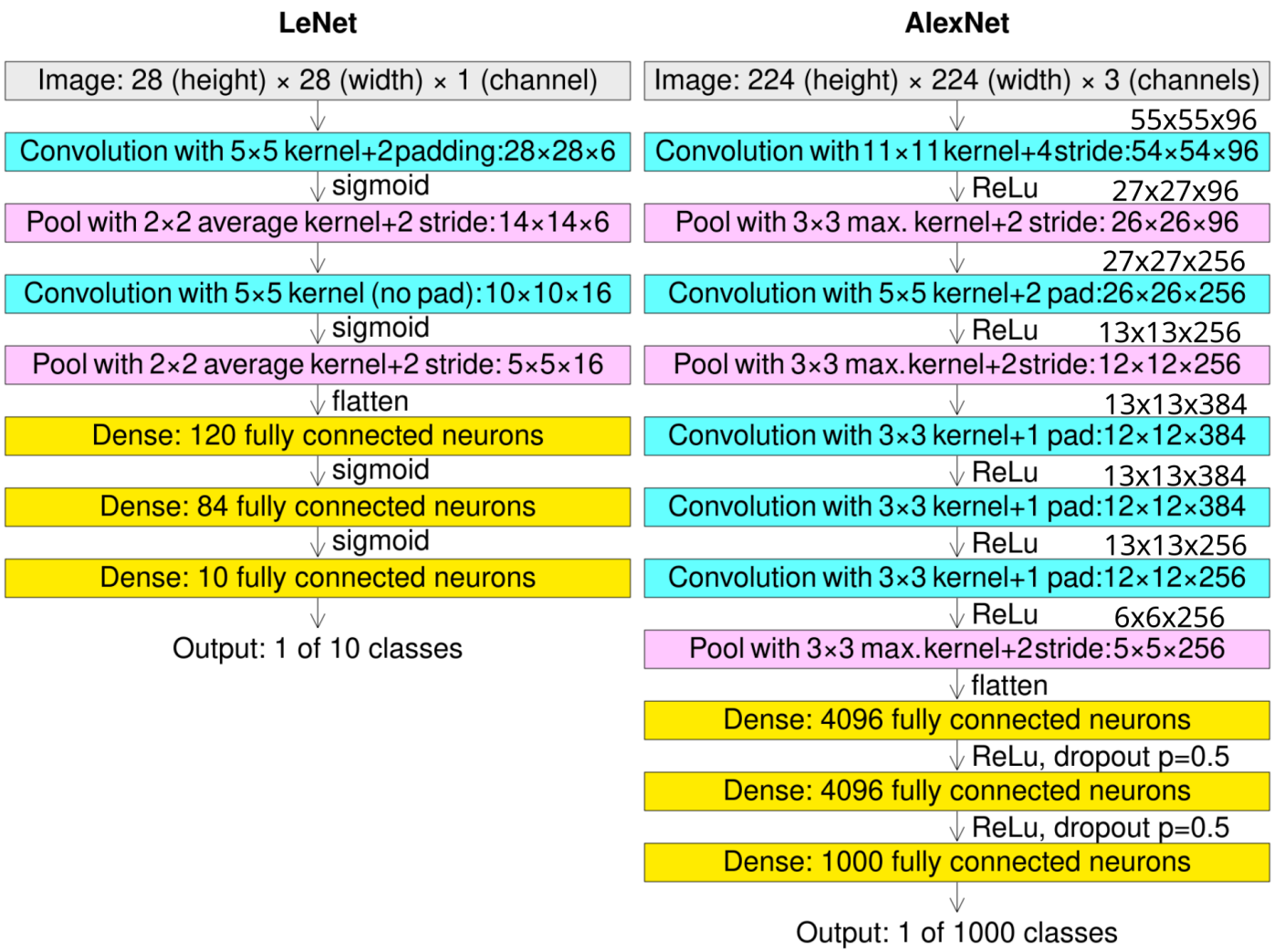
Partición estándar: $1\,281\,167$ de training, $50\,000$ de validación y $100\,000$ de test (con etiqueta de clase -1)

In [ ]:
```python
import numpy as np; import datasets
ds = datasets.load_dataset("ILSVRC/imagenet-1k"); ds
```

Out[ ]:
```
DatasetDict({
    train: Dataset({
        features: ['image', 'label'],
        num_rows: 1281167
    })
    validation: Dataset({
        features: ['image', 'label'],
        num_rows: 50000
    })
    test: Dataset({
        features: ['image', 'label'],
        num_rows: 100000
    })
})
```

6 AlexNet

AlexNet (2012): CNN para ImageNet que inició la revolución del aprendizaje profundo en el ILSVRC 2012



7 Dropout

Dropout: capa de regularización de las salidas de otra capa (entradas para dropout)

- Entrenamiento: anula cada entrada con una probabilidad dada p; las no anuladas se re-escalan con $\dfrac{1}{1-p}$
- Evaluación: actúa como la función identidad

Ejemplo: dropout aplicado a $\mathbf{X} = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$ en los siguientes casos

1. $p = 0.1,$ asumiendo que no anula ninguna entrada: $\mathrm{Dropout}(\mathbf{X}, 0.1) = \begin{pmatrix} 1 \\ 4 \end{pmatrix} \dfrac{1}{0.9} = \begin{pmatrix} 1.1111 \\ 4.4444 \end{pmatrix}$

2. $p = 0.5,$ asumiendo que anula la primera entrada: $\mathrm{Dropout}(\mathbf{X}, 0.5) = \begin{pmatrix} 0 \\ 4 \end{pmatrix} \dfrac{1}{0.5} = \begin{pmatrix} 0 \\ 8 \end{pmatrix}$

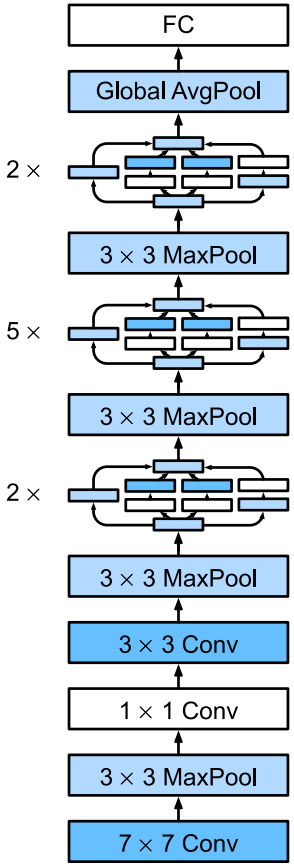
In [ ]:
```python
import torch; torch.random.manual_seed(23); X = torch.tensor([1, 4], dtype=torch.float)
print(torch.nn.Dropout(p=0.1)(X), torch.nn.Dropout(p=0.5)(X))
```

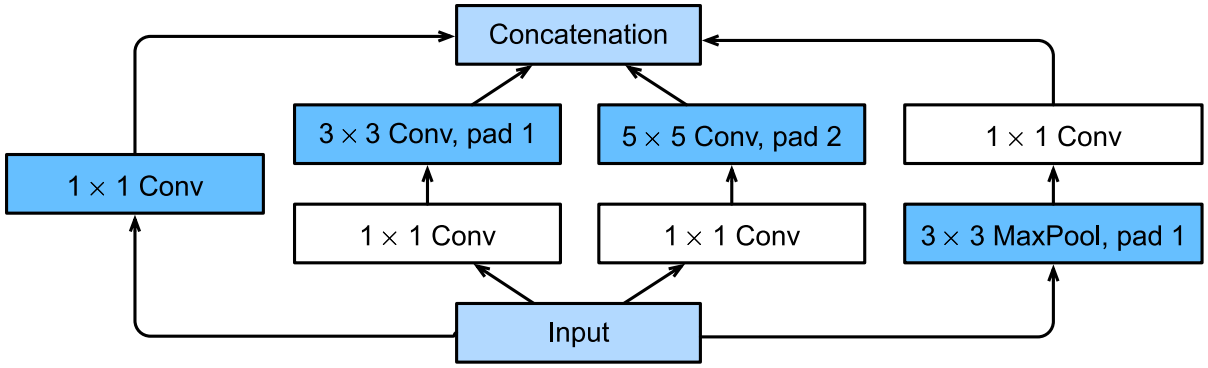
```
tensor([1.1111, 4.4444]) tensor([0., 8.])
```

8 Inception

Inception (2014): CNN de Google que introduce la meta-arquitectura típica de las CNNs modernas

- **Stem (raíz):** dos o tres capas convolucionales que extraen características de bajo nivel
- **Body (cuerpo):** subred de **bloques convolucionales** repetidos
- **Head (cabeza):** transforma la salida del cuerpo según la tarea a abordar (clasificación, segmentación, etc.)





Inception v3 (2015): Rethinking the Inception Architecture for Computer Vision

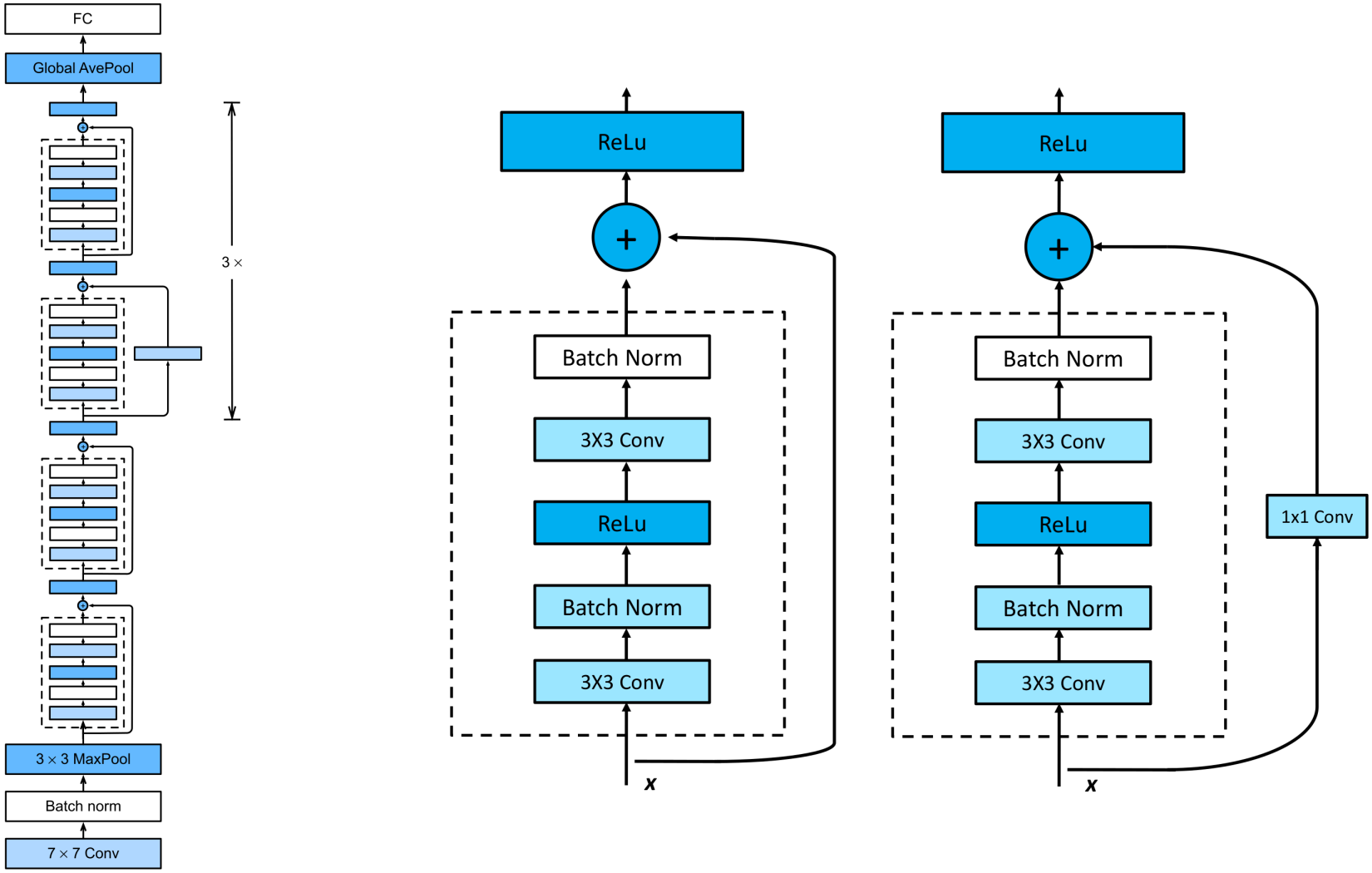
In [ ]:
```
import matplotlib.pyplot as plt; import urllib; from PIL import Image; import torch
import timm; from timm.data import resolve_data_config; from timm.data.transforms_factory import create_transform
model = timm.create_model('inception_v3', pretrained=True); model.eval()
config = resolve_data_config({}, model=model); transform = create_transform(**config)
url, file = ("https://github.com/pytorch/hub/raw/master/images/dog.jpg", "dog.jpg")
urllib.request.urlretrieve(url, file); img = Image.open(file).convert('RGB')
tensor = transform(img).unsqueeze(0); plt.xticks([]); plt.yticks([]); plt.imshow(img);
```

In [ ]:
```
probs = torch.nn.functional.softmax(model(tensor)[0], dim=0)
url, file = ("https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt", "imagenet_classes.txt")
urllib.request.urlretrieve(url, file); cats = [s.strip() for s in open("imagenet_classes.txt", "r").readlines()]
i = probs.argmax().item(); print(f'{cats[i]} {probs[i].item():.2%} ')
```

```
Samoyed 83.71%
```

9 ResNet

ResNet (2015): CNN de Microsoft que introduce **conexiones residuales** para entrenar modelos muy profundos



ResNet (2015): Deep Residual Learning for Image Recognition

In [ ]:
```
import matplotlib.pyplot as plt; import urllib; from PIL import Image; import torch
import timm; from timm.data import resolve_data_config; from timm.data.transforms_factory import create_transform
model = timm.create_model('resnet18', pretrained=True); model.eval()
config = resolve_data_config({}, model=model); transform = create_transform(**config)
url, file = ("https://github.com/pytorch/hub/raw/master/images/dog.jpg", "dog.jpg")
urllib.request.urlretrieve(url, file); img = Image.open(file).convert('RGB')
tensor = transform(img).unsqueeze(0); plt.xticks([]); plt.yticks([]); plt.imshow(img);
```

In [ ]:
```
probs = torch.nn.functional.softmax(model(tensor)[0], dim=0)
url, file = ("https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt", "imagenet_classes.txt")
urllib.request.urlretrieve(url, file); cats = [s.strip() for s in open("imagenet_classes.txt", "r").readlines()]
i = probs.argmax().item(); print(f'{cats[i]} {probs[i].item():.2%} ')
```

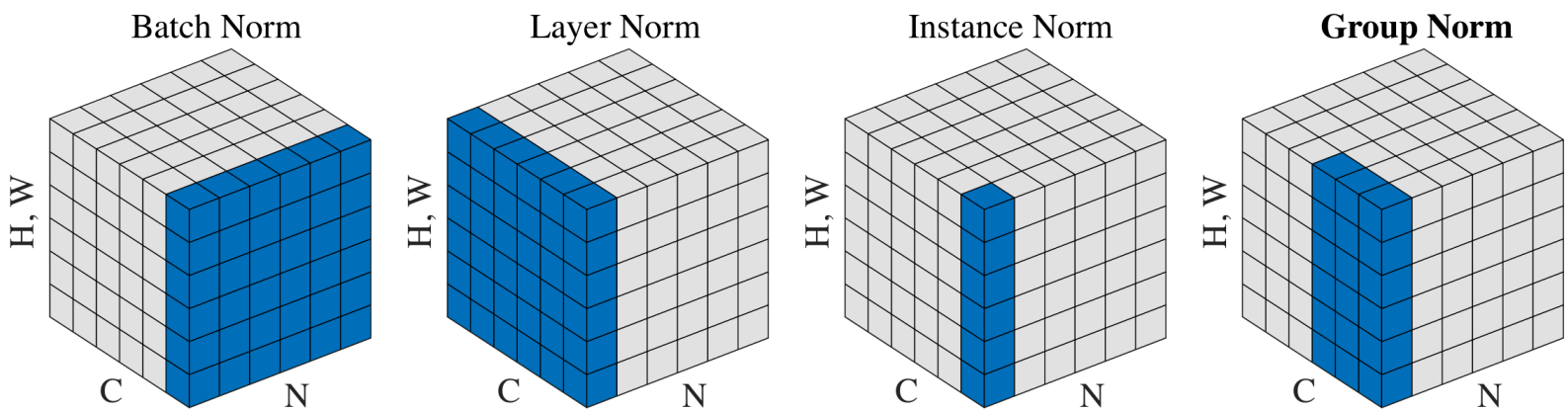
```
Samoyed 99.96%
```

10 Capas de normalización

Objetivo: añadir capas extra para evitar el problema de los gradientes que desaparecen o explotan en redes profundas

Batch/layer/instance/group norm: $\gamma = 1$ y $\beta = 0$ pueden aprenderse; $\epsilon = 10^{-5}$ por omisión

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}}\,\gamma + \beta$$



BatchNorm (2015): Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

InstanceNorm (2016): Instance Normalization: The Missing Ingredient for Fast Stylization

LayerNorm (2016): Layer Normalization

GroupNorm (2018): Group Normalization

Ejemplo batch norm: entrada $\mathbf{X}\, N \times C \times x_h \times x_w$ y salida $\mathbf{Y}\, N \times C \times x_h \times x_w$

$$\mathbf{X} = [[[0],[1]],[[2],[3]]]$$

$$\mathbf{Y} = \mathrm{BatchNorm2d}(\mathbf{X}) = [[[-1],[-1]],[[1],[1]]]$$

In [ ]:
```
import torch; X = torch.tensor([ [ [[0]], [[1]] ], [ [[2]], [[3]] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 2, 1, 1]) tensor([[[[0.]],[[1.]]],[[[2.]],[[3.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.BatchNorm2d(num_features=2, eps=0., momentum=0., affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 2, 1, 1]) tensor([[[[-1.]],[[-1.]]],[[[1.]],[[1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.transpose(0, 1).flatten(start_dim=1)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 2]) tensor([[0.,2.],[1.,3.]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=1).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[1.],[2.]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=1, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[1.],[1.]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).transpose(0, 1).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Ejemplo layer norm: entrada $\mathbf{X}\, N \times C \times x_h \times x_w$ y salida $\mathbf{Y}\, N \times C \times x_h \times x_w$

$$\mathbf{X} = \left[\left[\begin{bmatrix} 0 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \end{bmatrix} \right] \right]$$

$$\mathbf{Y} = \mathrm{LayerNorm}(\mathbf{X}) = \left[\left[\begin{bmatrix} -1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right] \right]$$

In [ ]:
```
import torch; X = torch.tensor([ [ [[0], [2]], [[2], [0]] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 2, 2, 1]) tensor([[[[0.],[2.]],[[2.],[0.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.LayerNorm(X.shape[1:], eps=0., elementwise_affine=False, bias=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 2, 2, 1]) tensor([[[[-1.],[1.]],[[1.],[-1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.flatten(start_dim=1)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 4]) tensor([[0.,2.,2.,0.]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=1).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[1.]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=1, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[1.]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Ejemplo instance norm: entrada $\mathbf{X}\, N \times C \times x_h \times x_w$ y salida $\mathbf{Y}\, N \times C \times x_h \times x_w$

$$\mathbf{X} = \left[\left[\left[\begin{matrix} 0 & 2 \\ 2 & 0 \end{matrix}\right]\right]\right]$$

$$\mathbf{Y} = \mathrm{InstanceNorm2d}(\mathbf{X}) = \left[\left[\left[\begin{matrix} -1 & 1 \\ 1 & -1 \end{matrix}\right]\right]\right]$$

In [ ]:
```
import torch; X = torch.tensor([ [ [ [0, 2], [2, 0] ] ] ], dtype=float)
print(X.shape, str(X).replace('\n',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[0., 2.],          [2., 0.]]]], dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.InstanceNorm2d(num_features=1, eps=0., momentum=0., affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[-1.,  1.],          [ 1., -1.]]]], dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.flatten(start_dim=2)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 4]) tensor([[[0.,2.,2.,0.]]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=2).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[1.]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=2, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[1.]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Ejemplo group norm: entrada $\mathbf{X}\, N \times C \times x_h \times x_w$ y salida $\mathbf{Y}\, N \times C \times x_h \times x_w$

$$\mathbf{X} = [[[0],[1],[2],[3]]]$$

$$\mathbf{Y} = \mathrm{GroupNorm}(\mathbf{X}, 2) = [[[-1],[1],[-1],[1]]]$$

In [ ]:
```
import torch; X = torch.tensor([ [ [[0]], [[1]], [[2]], [[3]] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 4, 1, 1]) tensor([[[[0.]],[[1.]],[[2.]],[[3.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.GroupNorm(num_groups=2, num_channels=4, eps=0., affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 4, 1, 1]) tensor([[[[-1.]],[[1.]],[[-1.]],[[1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.reshape(X.shape[0], 2, X.shape[1]//2, *X.shape[2:]).flatten(start_dim=2)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 2, 2]) tensor([[[0.,1.],[2.,3.]]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=2).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[0.5000],[2.5000]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=2, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[0.5000],[0.5000]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

RMSNorm: Root Mean Square Layer Normalization se aplica sobre las últimas dimensiones del batch

$$y_i = \frac{x_i}{\mathrm{RMS}(x)} * \gamma_i, \quad \text{donde} \quad \mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

Ejemplo: entrada $\mathbf{X}\, N \times C \times x_h \times x_w$ y salida $\mathbf{Y}\, N \times C \times x_h \times x_w$ sobre la última dimensión

$$\mathbf{X} = \left[\left[\left[\begin{matrix} 1 & 1 \\ 2 & 2 \end{matrix}\right]\right]\right] \qquad \mathbf{Y} = \mathrm{RMSNorm}(\mathbf{X}, (2,)) = \left[\left[\left[\begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix}\right]\right]\right]$$

In [ ]:
```
import torch; X = torch.tensor([ [ [ [1, 1], [2, 2] ] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[1.,1.],[2.,2.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.RMSNorm(normalized_shape=X.shape[-1:], eps=0., elementwise_affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[1.,1.],[1.,1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.flatten(start_dim=-1)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[1.,1.],[2.,2.]]]],dtype=torch.float64)
```

In [ ]:
```
X_rms = X_flat.square().mean(dim=-1).reshape(-1, 1).sqrt()
print(X_rms.shape, str(X_rms).replace('\n','').replace(' ',''))
```

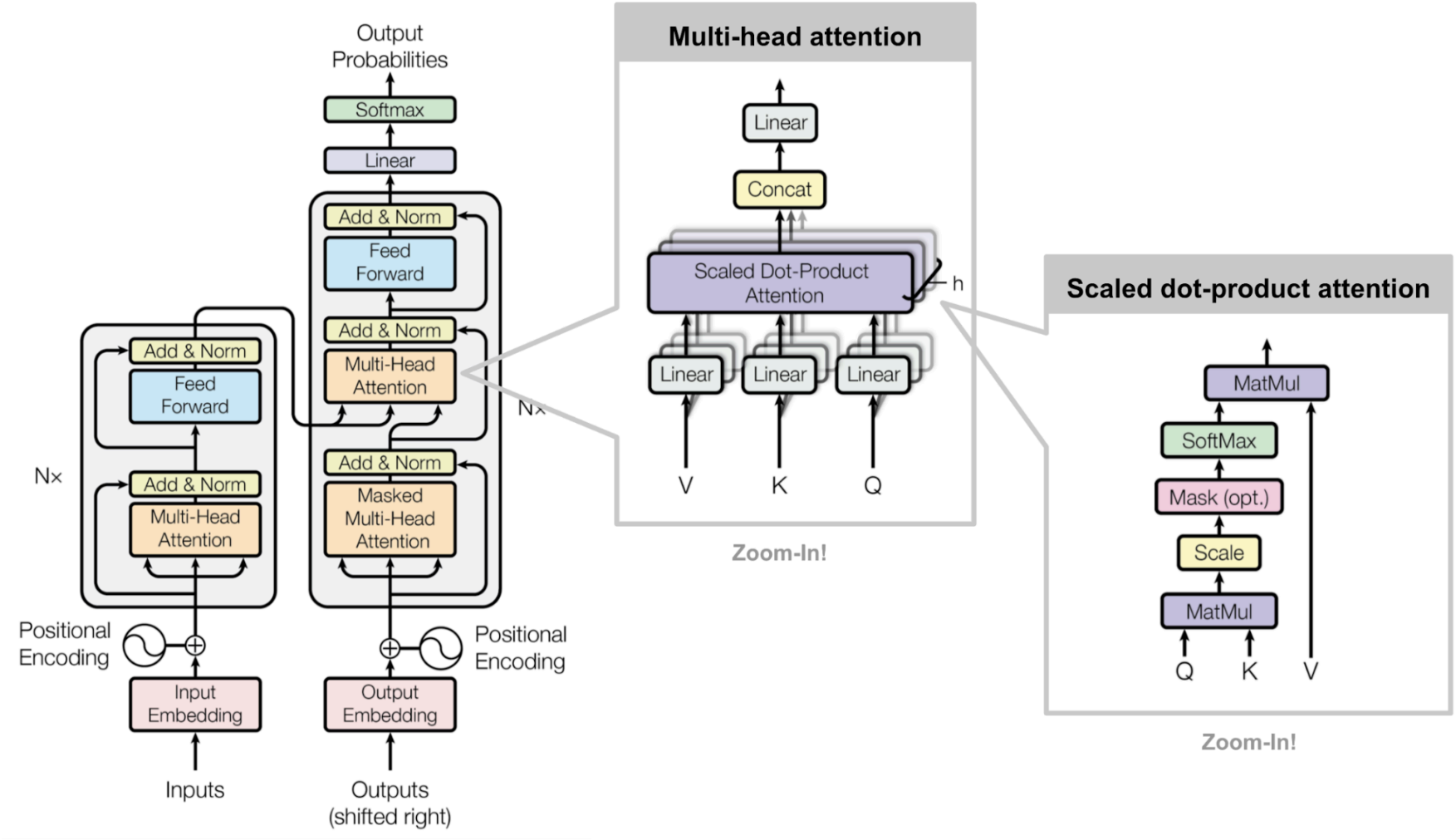
```
torch.Size([2, 1]) tensor([[1.],[2.]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = (X_flat / X_rms).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

11 Transformer

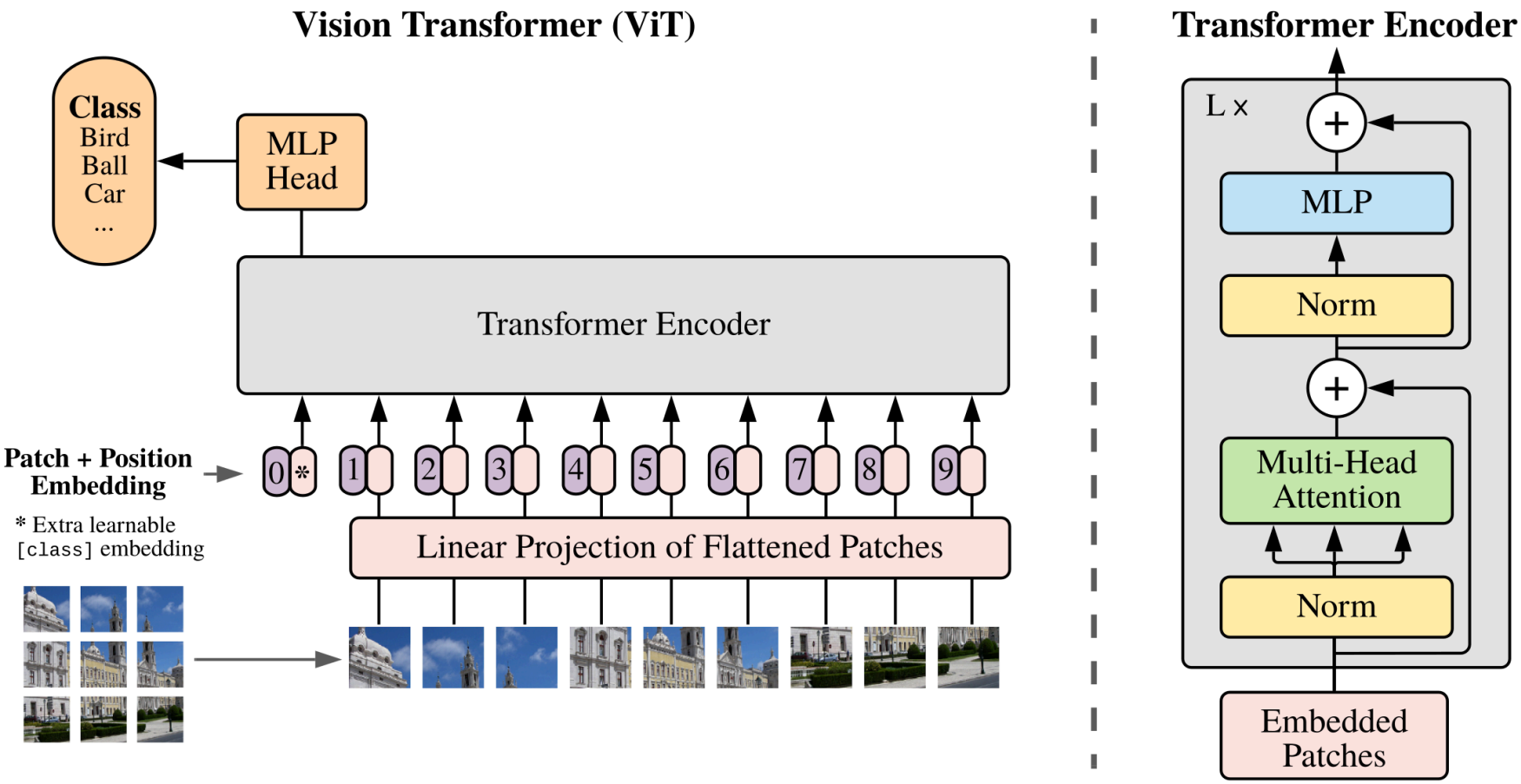
Transformer (2017): arquitectura de Google para traducción automática basada en el **mecanismo de atención**



3blue1brown: ver Transformers, the tech behind LLMs y Attention in transformers, step-by-step

12 Vision Transformer

Vision Transformer (2020): adaptación del Transformer para visión que rivaliza con las CNNs



SOTA en ImageNet: ver ficheros de resultados de Ross Wightman